

Benchmarking EML based Neural Networks

Hrishikesh K Haritas · Carnegie Mellon University Code: `emlkit`

ABSTRACT

Odrzywołek (2026) showed that the single binary operator $\text{eml}(x, y) = e^x - \ln y$, together with the constant 1, generates every elementary function. We ask whether that operator can serve as the only primitive of a *trainable, interpretable* regression model. EML networks are layered graphs of EML nodes with affine arguments. They are trained by gradient descent, then pruned and snapped to exact exponents. Training needs four ingredients: least-squares solution of the output node's argument in log space, successive halving over restarts, an annealed pull of exponents towards multiples of $\frac{1}{2}$, and Levenberg–Marquardt verification of each pruned or snapped structure.

On the 99 Feynman equations, EML networks recover the exact law for 82% of the held-out problems without trigonometric functions (65% of all problems, against 26% for genetic programming). In distribution they match an MLP: $R^2 > 0.999$ on 92% of problems against 55%. Out of distribution they keep $R^2 > 0.99$ on 81% of problems, against 5% for the MLP and 0% for gradient boosting. The learned formulas are cheap to run: a median $147\times$ faster than a 2×128 MLP, and an MLP needs a median $10\times$ more parameters to match them on the Feynman problems. Training, however, is a median $184\times$ slower. We also report where the method fails: trigonometric laws, sign-changing products, exponential blow-ups on small noisy data, and data sets without a compact law. Every claim was tested against thresholds fixed before the results. We release `emlkit`, a scikit-learn estimator plus a compiler that turns any elementary expression into a pure EML tree.

1 Introduction

Symbolic regression looks for a formula, not just a predictor. A formula can be read, checked against theory, and trusted outside the range of the data. Most practical systems search over expression trees built from a hand-picked operator set ($+$, $-$, $\times$, $\div$, $\exp$, $\log$, $\sin$, ...) with genetic programming. Odrzywołek's result offers a different starting point. A single operator, $\text{eml}(x, y) = e^x - \ln y$, is complete for elementary functions in the way NAND is complete for Boolean logic. A model built from one differentiable primitive can be trained end to end by gradient descent, and every trained model is still a formula.

Pure EML trees are deep, though. Our compiler needs 23 leaves for $x \cdot y$ and 264 for $\sin x$, and gradients through nested $\exp/\ln$ are badly conditioned. This paper makes the idea practical and tests it against concrete, pre-declared claims ([Table 1](#)). We contribute:

- **EML networks:** EML nodes with affine arguments and typed units, so that products, powers, radicals and exponentials of monomials need one to three nodes instead of dozens (§2).

- A **training recipe** that turns near-misses into exact formulas: log-space variable projection, successive halving, annealed quantisation, Levenberg–Marquardt verification of structural edits, and an Occam curriculum with BIC selection (§3).
- An **evaluation** on 99 Feynman equations (with out-of-distribution tests), 15 real physics data sets and 34 tabular data sets. It covers accuracy, explainability, comparison with neural networks, compute, and ablations, with a scorecard of 18 pre-declared claims (§5).
- **emlkit**, an open-source scikit-learn estimator and a pure-EML compiler, verifier and shortest-tree search.

2 EML networks

An EML network is a layered directed acyclic graph. Its input features are $\mathbf{z}_0 = [\mathbf{x}, \ln |\mathbf{x}|]$; the fixed log-features are themselves EML nodes, since $\ln \mathbf{x} = \mathbf{1} - \text{eml}(\mathbf{0}, \mathbf{x})$. Every unit sees all features computed before it through affine maps $\mathbf{a} = \mathbf{W}_a \mathbf{z} + \mathbf{b}_a$ and $\mathbf{c} = \mathbf{W}_c \mathbf{z} + \mathbf{b}_c$, and is one of three types:

$$E: \text{eml}(\mathbf{a}, 1) = e^{\mathbf{a}}, \quad L: 1 - \text{eml}(\mathbf{0}, \mathbf{c}) = \ln |\mathbf{c}|, \quad M: \text{eml}(\mathbf{a}, \mathbf{c}) = e^{\mathbf{a}} - \ln |\mathbf{c}|.$$

E and L are the full operator with one argument fixed to the constant 1, or to $0 = \ln 1$. Fixing an argument removes a redundant degree of freedom. Because $\ln |\mathbf{x}|$ is an input feature, a monomial $C \prod_j x_j^{p_j}$ is a single E unit, $\exp(\sum_j p_j \ln x_j + \ln C)$. The output is either an affine read-out of the last layer, or itself an E node $\mathbf{y} = \mathbf{s} \exp(\mathbf{w}^\top \mathbf{z} + \mathbf{b})$ with a fixed sign $\mathbf{s}$. In the second case the network models $\ln |\mathbf{y}|$ as a *linear* function of its features, so products of factors become sums (Figure 1). Pruned weights are exactly zero and snapped weights are exact rationals, so a trained network is a closed-form expression that SymPy simplifies directly.

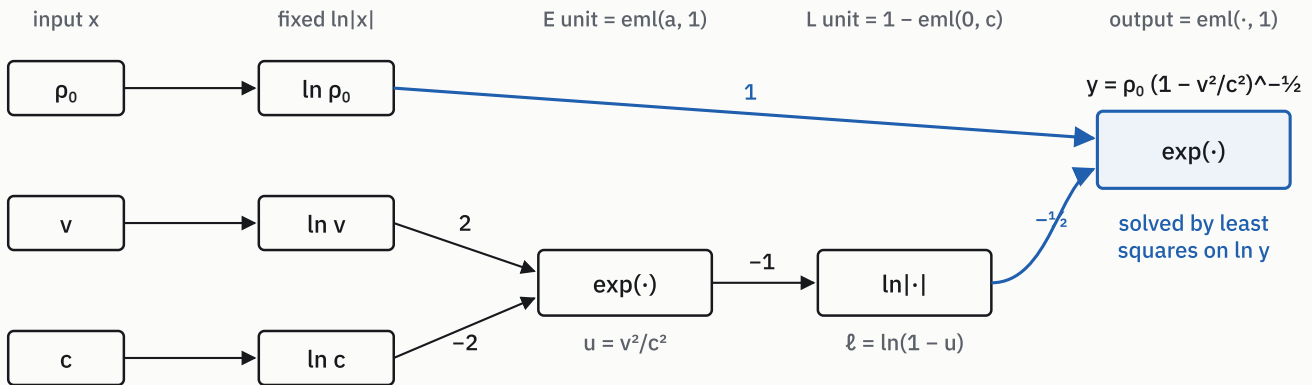


Figure 1. The network EML-Net finds for the relativistic density $\rho_0/\sqrt{1 - v^2/c^2}$ (Feynman II.13.23). Black arrows carry weights learned by gradient descent and snapped to multiples of $1/2$. The output is itself an EML node, so the weights on its incoming (blue) arrows are linear coefficients of $\ln y$ and are solved exactly by least squares at every step. Gradient descent only has to discover the two inner units.

The table below lists the shallowest architecture that represents each family of laws exactly. A star marks a log-space output node.

FAMILY	EXAMPLE (FEYNMAN)	ARCHITECTURE
power law · exp(linear)	$q_1 q_2 / (4\pi\epsilon r^2)$	★ (no hidden unit)
· exp(monomial)	$n_0 e^{-mgx/kT}$	E1★
· affine ^p	$\sigma/(\epsilon(1+\chi))$	L1★
sum of monomials	$Gm_1 m_2 (1/r_2 - 1/r_1)$	E2, E3
· (1 ± monomial) ^p	$mo/\sqrt{(1 - v^2/c^2)}$	E1-L1★
· (Σ monomials) ^p	$\sqrt{(x^2 + y^2 + z^2)}$	E3-L1★
· (e ^{monomial} − 1) ^p	$\hbar\omega/(e^{\hbar\omega/kT} - 1)$	E1-E1-L1★
· (Σ (difference) ²) ^p	$Gm_1 m_2 / \Sigma(x_2 - x_1)^2$	L3-E3-L1★

3 Training

All restarts of an architecture are trained in one batched tensor computation. Each parameter carries a pruning mask and a snapping mask.

1. **Variable projection.** The read-out is never trained by gradient descent. At every step it is solved by ridge least squares, on y for linear outputs and on $\ln|y|$ for log-space outputs, with snapped entries held fixed. Gradients only shape the EML features (Golub & Pereyra, 1973). For a log-space output this also solves every exponent attached to the input logs.
2. **Structured initialisation and successive halving.** E units start from sparse integer exponents. In half the restarts, units fed by earlier units start as $\exp(\pm u)$ or $\ln|1 \pm u|$. Training starts with 4× the target number of restarts and keeps the better half twice.
3. **Annealed quantisation.** In the second half of dense training a penalty $\rho(t) \sum (w - \frac{1}{2} \text{round}(2w))^2$ pulls argument weights towards multiples of $\frac{1}{2}$.
4. **Prune, snap, verify.** Weights below a magnitude threshold, and read-out terms that contribute less than a fraction of $\text{std}(y)$, are removed. Weights near a multiple of $\frac{1}{2}$ are frozen there. After each edit the free parameters are refit with Adam, then with Levenberg–Marquardt, and a restart is rolled back if its validation error rose by more than 10%. Levenberg–Marquardt is what makes an exact structure *look* exact: started at the true solution of II.13.23, Adam fine-tuning drifts to $\text{NMSE} \approx 10^{-7}$, while LM returns to 10^{-19} . Restarts are independent, so perturbing parameter slot k in every restart at once yields column k of every Jacobian in one forward-mode pass.
5. **Occam curriculum.** Eleven architectures, from the plain power law ★ to a wide E4-L2-E2 with skip read-out, are tried in order of complexity. Log-space stages are used only when y has a constant sign. The search stops early

when a candidate fits the validation data to $\text{NMSE} < 10^{-12}$. The final model minimises $\text{BIC} = n \ln(\text{NMSE} + 10^{-14}) + k \ln n$ over all candidates.

4 Experimental setup

Feynman. All 99 equations of the AI Feynman database (Udrescu & Tegmark, 2020), with formulas and variable ranges from PMLB. For each problem we draw 1,000 training points from the original box, 2,000 test points from the same box (ID), and 2,000 points with every variable in $[\text{hi}, 2 \cdot \text{hi} - \text{lo}]$ (OOD, pure extrapolation). Label noise is 0 or 1% of $\text{std}(y)$; test sets are noiseless. Recovery is the SRBench criterion: after rounding constants to three decimals, $\text{model} - \text{truth}$ or $\text{model} / \text{truth}$ simplifies to a constant. Eight problems were used to develop the method and are reported separately as the dev set.

Real data. Fifteen physics data sets of measured or simulated laws, from PMLB's *first principles* collection (Cranmer, 2023) and Nikuradse's pipe-flow measurements. Thirty-four general tabular sets: five UCI classics (yacht, airfoil, concrete, energy, auto-MPG) and every small real-world PMLB regression set with ≤ 12 features and $\leq 5,000$ rows, excluding synthetic generators. Metric: pooled out-of-fold R^2 from 5-fold CV (leave-one-out for $n < 20$).

Baselines (default settings, no test-set tuning): Ridge regression; an MLP with two hidden layers of 128 ReLU units, standardised inputs and target, and early stopping; histogram gradient boosting; random forest; GPLEarn genetic programming (population 2,000, 30 generations). EML uses the library defaults (24 restarts per stage, 1,000-step budget per stage). Every run uses one CPU core on Modal with identical package versions.

Compute. On one split per problem (Feynman with 1% noise; tabular with a 75/25 split) we compare EML with MLPs of nine sizes and with gradient boosting in parameters, inference latency and training time (§5.4).

5 Results

Table 1 states each claim, the evidence and the verdict, computed mechanically from the thresholds declared in `CLAIMS.md` before the full results existed.

Table 1. Claims scorecard.

#	CLAIM		
C1	EML recovers exact laws		
C2	at least as good as GP symbolic regression		
C3	matches an MLP in distribution		
C4	extrapolates far better than MLP / GBM		
C5	survives 1% label noise		
C6	models are readable		
C7	best model on some real data sets	15 data sets: bode, kepler, le	
C8	accurate while explainable (physics data)		
C9	comparable to a NN (tabular data)		
C10	every training component matters	wide -30 pts; noVP -27 pts; noLM -19 pts; untyped -19	
C11	compiler is exact		
C13	same accuracy with far fewer parameters (Feynman)	matching MLP needs	
C14	same accuracy with fewer parameters (tabular)	matching MLP needs	
C15	cheaper inference		
C16	beats a parameter-matched NN		
C17	training cost comparable to an MLP		
C18	pruning keeps accuracy		
C12	limitation: trigonometric laws		

5.1 Feynman equations

Table 2. 99 Feynman problems without noise (top) and with 1% label noise (bottom).

METHOD	EXACT RECOVERY	$R^2_{ID} > 0.999$	$R^2_{OOD} > 0.99$	OOD NMSE $< 10^{-9}$	MEDIAN SIZE	MEDIAN S
EML	65%	92%	81%	68%	14	58
GPLearn	26%	36%	40%	23%	17	16
MLP	–	55%	5%	0%	–	2
GBM	–	3%	0%	0%	–	1
RF	–	1%	0%	0%	–	1
Linear	–	0%	3%	0%	–	0

METHOD	EXACT RECOVERY	$R^2_{ID} > 0.999$	$R^2_{OOD} > 0.99$	$OOD\ NMSE < 10^{-9}$	MEDIAN SIZE	MEDIAN S
EML	34%	94%	81%	0%	17	321
GPLearn	26%	35%	40%	23%	16	19
MLP	–	54%	4%	0%	–	3
GBM	–	2%	0%	0%	–	1
RF	–	1%	0%	0%	–	1
Linear	–	0%	3%	0%	–	0

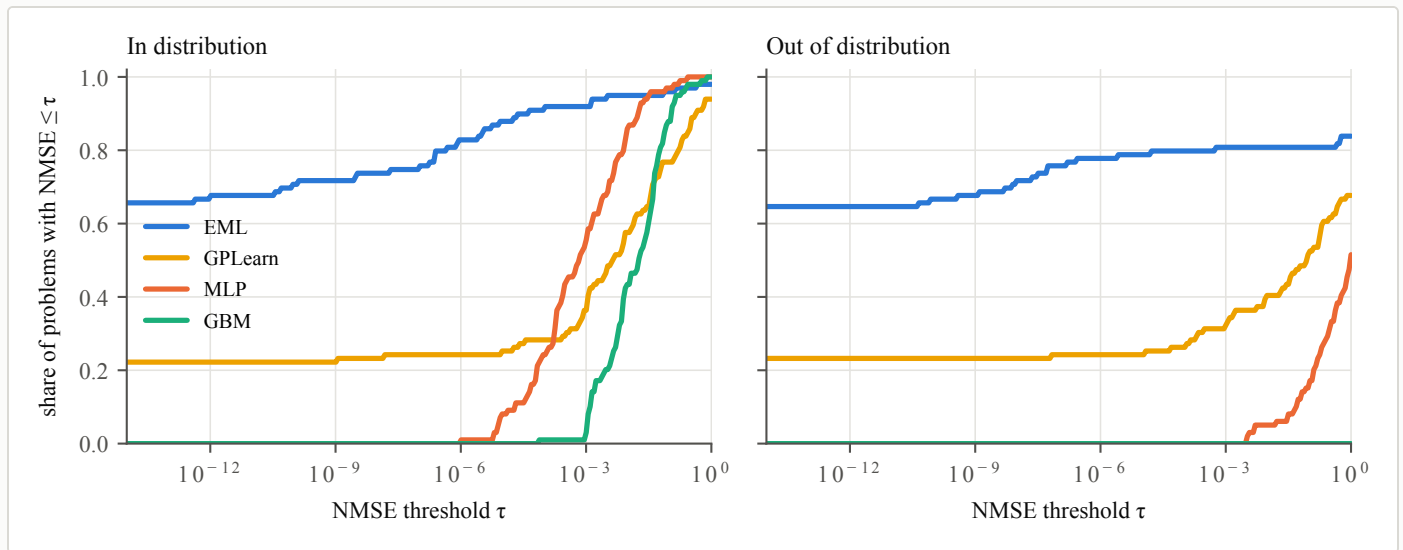


Figure 2. Share of the 99 problems whose test NMSE is below τ . In distribution the MLP is close to EML at moderate thresholds, but only exact formulas reach the left of the plot. Out of distribution, the gap between laws and fitted surfaces opens over the whole range.

EML recovers 65% of all problems exactly (66% of the held-out set, 50% of the dev set, and 82% of held-out problems without trigonometric functions). GPLearn recovers 26%. Every method that fits well in distribution looks similar there, but extrapolation separates them. 68% of EML models stay exact ($NMSE < 10^{-9}$) far outside the training box, against 0% for the MLP and 0% for gradient boosting. With 1% label noise EML still recovers 34% of the laws, and keeps $OOD\ R^2 > 0.99$ on 81% of problems.

5.2 Real data

Table 3. Physics data: 5-fold CV R^2 (leave-one-out for $n < 20$); best value per row in green. The last column is the EML formula fit to all data.

DATA SET	N	EML	GPLEARN	MLP	GBM	RF	LINEAR
absorption	14	-300.181	-16.949	0.974	0.711	0.885	0.625
bode	8	0.982	-175252777639.843	-551.294	-0.306	0.554	-19304.546
hubble	32	0.709	0.208	0.730	0.862	0.816	0.697
ideal_gas	30	0.378	0.537	0.879	0.674	0.632	0.830
kepler	6	1.000	0.818	0.865	-0.440	0.214	0.905
leavitt	26	0.951	0.933	0.924	0.645	0.896	0.915
newton	30	-13.851	0.977	-10.774	0.861	0.810	-0.841
planck	100	1.000	-0.013	0.460	0.785	0.812	0.203
rydberg	50	1.000	0.987	0.999	0.997	0.993	0.967
schechter	27	0.996	0.828	0.989	0.753	0.889	0.744
supernovae_zg	243	-6.703	0.851	0.939	0.998	0.999	0.553
supernovae_zr	236	-2233357029793081088.000	0.867	0.998	0.993	0.993	0.499
tully_fisher	18	0.924	0.838	0.455	0.775	0.826	0.860
nikuradse_1	362	0.998	0.939	0.999	0.998	0.998	0.653
nikuradse_2	362	0.973	0.825	0.977	0.952	0.966	-0.003

Table 4. 34 general tabular data sets: median 5-fold CV R^2 and mean rank (1 = best).

METHOD	MEDIAN R^2	MEAN RANK
EML	0.743	3.32
GPLearn	0.637	4.50
MLP	0.774	2.94
GBM	0.717	3.82
RF	0.763	3.09
Linear	0.638	3.32

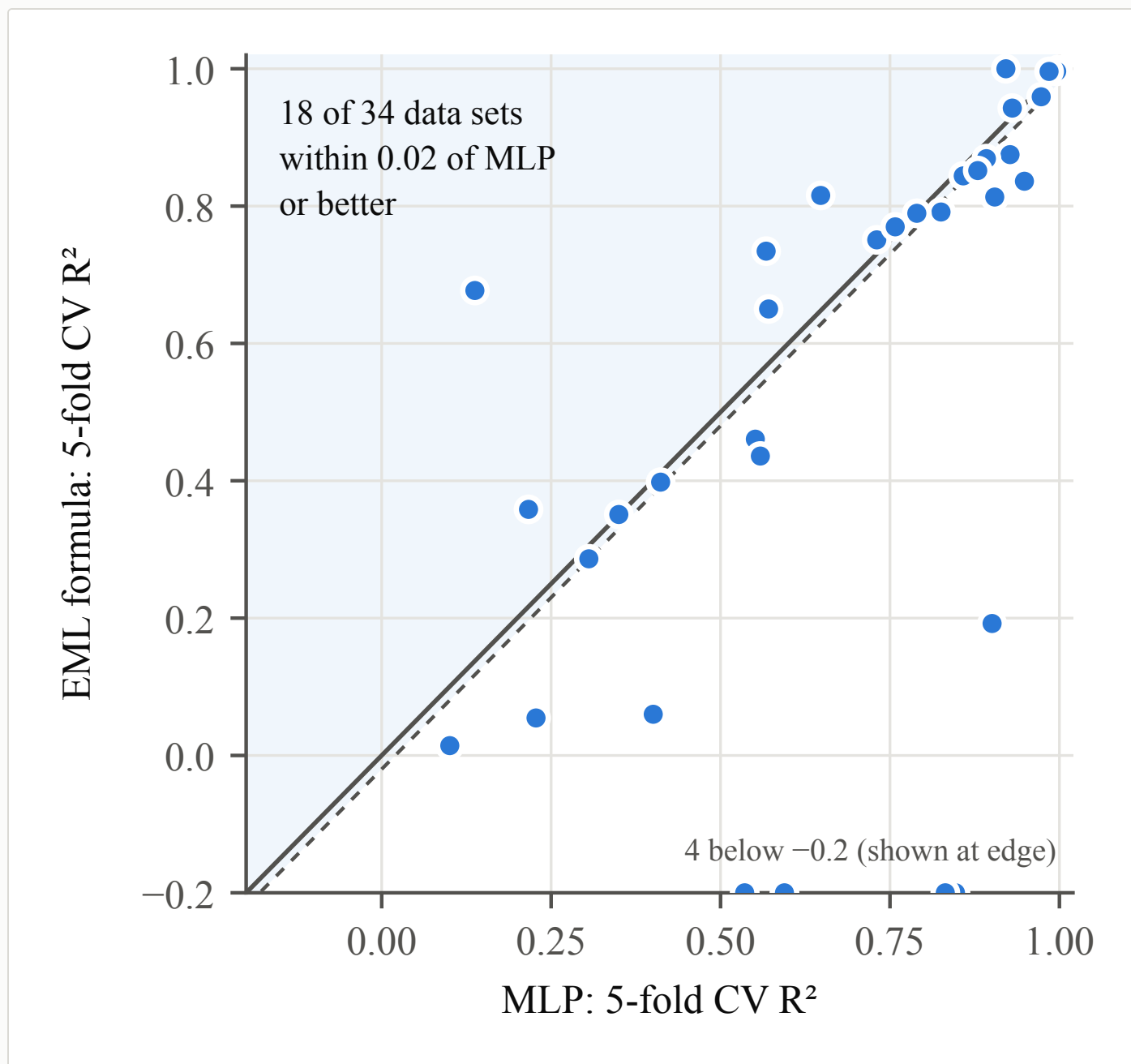


Figure 3. EML closed-form models against the MLP on general tabular data. Points above the dashed line are within 0.02 R^2 of the MLP or better.

5.3 Ablations

Table 5. Exact recovery on 37 held-out non-trigonometric Feynman problems (every other one, fixed list) when one component is removed.

VARIANT	RECOVERY	CHANGE
full method	81%	–
no curriculum (one wide net)	51%	-30 pts
no variable projection	54%	-27 pts
no Levenberg–Marquardt	62%	-19 pts
untyped M units only	62%	-19 pts
no log-space projection (linear output only)	70%	-11 pts
no annealed quantisation	70%	-11 pts
no successive halving	73%	-8 pts
read-out sees all features	81%	-0 pts

5.4 Compute

For each problem we also fit MLPs with one hidden layer of 1 to 128 units, plus the 2×128 baseline, and measured parameters, single-core NumPy inference time on 100k rows, and training time. EML formulas are tiny and fast to evaluate. The price is paid during training.

Table 6. Medians per task. *Params ratio*: parameters of the smallest MLP that matches EML's test R^2 within 0.005, divided by EML's (EML wins outright where no MLP matches). *vs. same-size MLP*: share of tasks where EML $\geq$ the smallest MLP with at least as many parameters.

TASKS	N	EML R^2	2×128 MLP R^2	EML PARAMS	PARAMS RATIO	VS. SAME-SIZE MLP	1
Feynman, 1% noise	99	1.000	0.999	8	10×	99%	
Tabular	34	0.759	0.743	10	4×	85%	

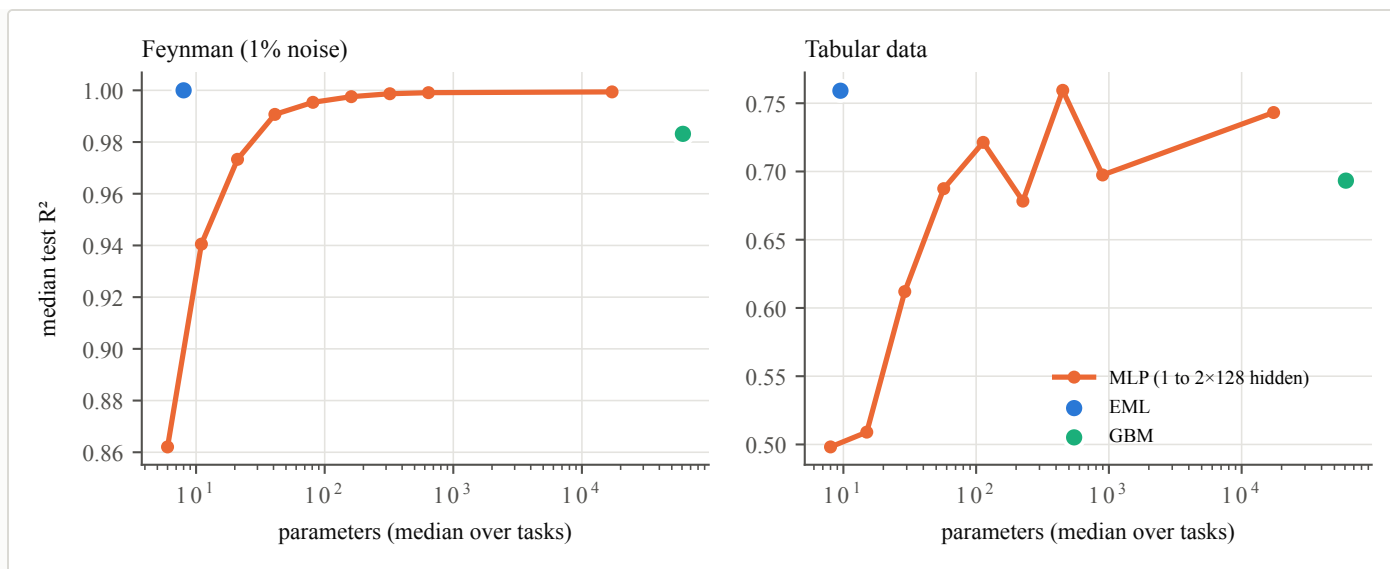


Figure 4. Median test R^2 against parameter count. The MLP curve has to grow by orders of magnitude to reach the accuracy of the EML formula on the Feynman problems; on tabular data the gap is smaller.

5.5 The pure EML compiler

`emlkit.tree` compiles any elementary SymPy expression into a tree over $\{\text{eml}, 1, x\}$ using textbook identities, for example $\ln x = \text{eml}(1, \text{eml}(\text{eml}(1, x), 1))$ and $a - b = \text{eml}(\ln a, e^b)$. It evaluates the tree with the principal complex logarithm and checks it against the source expression. The constructions are correct, not minimal. Exhaustive search confirms the shortest trees for $\exp$ and $\ln$, with 2 and 4 leaves. The sizes explain why we train shallow typed networks rather than pure trees: $x \cdot y$ already needs 23 leaves and $\sin x$ needs 264. Fractional powers of negative numbers can land on the other branch of the logarithm. We verify on domains where the source expression is real.

Table 7. Pure EML trees produced by the compiler (leaves, depth, distinct nodes when identical subtrees are shared) and maximum relative error at 64 random points.

EXPRESSION	LEAVES	DEPTH	SHARED NODES	MAX ERROR
$\exp(x)$	2	1	1	0e+00
$\log(x)$	4	3	3	2e-16
$x - y$	6	4	5	4e-16
$-x$	11	8	9	1e-16
$x + y$	16	10	14	3e-16
$x*y$	23	12	21	4e-16
x/y	13	8	12	2e-16
$x**y$	27	16	25	2e-15
$\text{sqrt}(x)$	54	25	41	3e-16
2	16	10	10	0e+00
pi	67	26	32	0e+00
I	42	20	26	3e-17
$\sin(x)$	264	71	94	9e-16
$\cos(x)$	220	55	85	9e-16
$\tanh(x)$	77	27	44	5e-16
$\exp(-x**2/2)/\text{sqrt}(2*\text{pi})$	328	68	139	2e-16
$x*y/(4*\text{pi}*z**2)$	231	52	125	8e-16

6 Limitations

- **Trigonometric laws.** Real-valued EML networks cannot express $\sin$ or $\cos$ compactly (§5.5). EML recovers 0% of the trigonometric Feynman problems. Complex-valued units, where $\ln(-1) = i\pi$, are the natural extension.
- **Sign-changing products.** Log-space stages need y of constant sign, and $\ln|\cdot|$ loses the sign of a factor. A law such as $n k T \ln(V_2/V_1)$ is therefore only approximated.
- **Training cost.** A fit takes seconds when an early curriculum stage is exact and minutes otherwise, typically two orders of magnitude more than an MLP (C17).
- **Blow-ups on small noisy data.** On a few small tabular sets a learned $\exp(\cdot)$ explodes on held-out folds ($\text{CV } R^2 < -1$). Clipping predictions to the training range would hide this, so we do not.
- **A missing stage.** A target that is a plain sum of logarithms (the *ideal_gas* set stores $\ln P$) is not represented by any strict curriculum stage, and EML fails on it.

- **Scope.** One seed per configuration, default hyper-parameters, and GPLearn as the only GP baseline. Stronger GP systems (PySR, Operon) would make a harder comparison.

7 Related work

Gradient-trained symbolic layers go back to product units (Durbin & Rumelhart, 1989) and the Equation Learner (Martius & Lampert, 2016; Sahoo et al., 2018). Kolmogorov–Arnold networks (Liu et al., 2024) learn univariate splines that are symbolified afterwards. Genetic programming dominates practice (Koza, 1992; Cranmer, 2023), and SRBench (La Cava et al., 2021) standardised its evaluation. AI Feynman (Udrescu & Tegmark, 2020) introduced the benchmark used here. EML networks differ in using one operator for every node and in solving the outer formula exactly by log-space variable projection.

8 Conclusion

A single operator is enough to build a trainable model whose fitted parameters *are* a formula. With the right optimisation scaffolding, gradient descent on EML networks recovers many physical laws exactly. Those laws keep their accuracy far outside the training range, where neural networks and tree ensembles fail. On data without a compact law the formulas remain readable, at a measured cost in accuracy. Code, data, raw results and this paper are reproducible with `emlkit`.

References

- Cranmer, M. (2023). Interpretable machine learning for science with PySR and SymbolicRegression.jl. arXiv:2305.01582.
- Durbin, R. & Rumelhart, D. (1989). Product units: a computationally powerful and biologically plausible extension to backpropagation networks. *Neural Computation* 1(1).
- Golub, G. & Pereyra, V. (1973). The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate. *SIAM J. Numer. Anal.* 10(2).
- Koza, J. (1992). *Genetic Programming*. MIT Press.
- La Cava, W. et al. (2021). Contemporary symbolic regression methods and their relative performance. NeurIPS Datasets and Benchmarks.
- Liu, Z. et al. (2024). KAN: Kolmogorov–Arnold Networks. arXiv:2404.19756.
- Martius, G. & Lampert, C. (2016). Extrapolation and learning equations. arXiv:1610.02995.
- Nikuradse, J. (1933). Strömungsgesetze in rauhen Röhren. VDI-Forschungsheft 361.
- Odrzywołek, A. (2026). All elementary functions from a single binary operator. arXiv:2603.21852.
- Sahoo, S., Lampert, C. & Martius, G. (2018). Learning equations for extrapolation and control. ICML.

Udrescu, S.-M. & Tegmark, M. (2020). AI Feynman: a physics-inspired method for symbolic regression. *Science Advances* 6(16).